

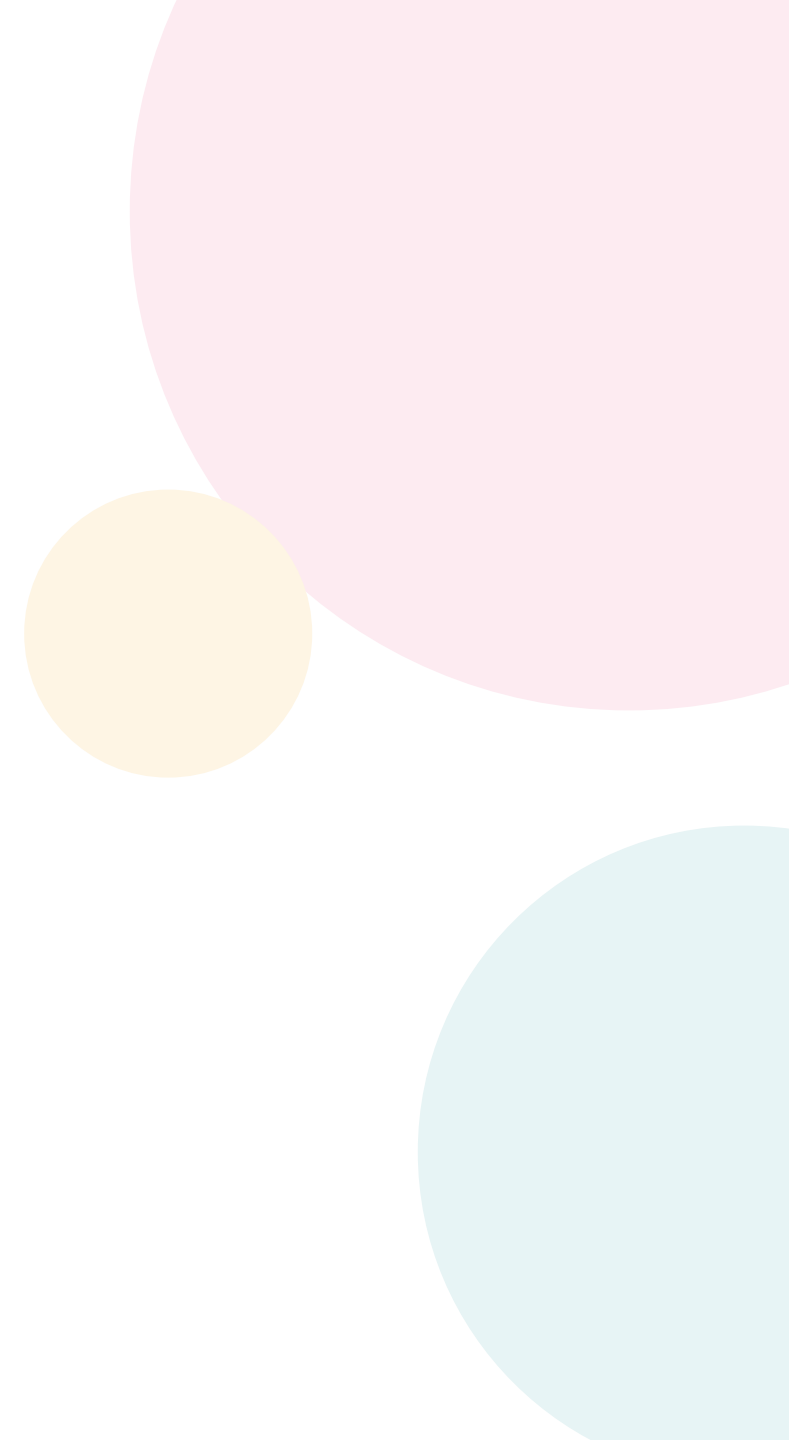
PROGRAMAÇÃO ORIENTADA A OBJETOS COM JAVA E WEB

Coleções em Java

Genéricos, Collections Framework e Programação Funcional

Prof. Dr. Antonio Marcos Selmini

selmini@fiap.com.br · Sistemas de Informação



AGENDA

Roteiro da aula

01

Coleções e o Collections Framework

02

Interface List e a classe ArrayList

03

Percorrendo e ordenando listas

04

Comparable, Comparator e equals()

05

Interface Set e interface Map

06

Exercícios e o tipo Record

Por que precisamos de coleções?

Array

- Tamanho fixo, definido na criação
- Não cresce em tempo de execução
- Poucas operações prontas
- Inserir/remover no meio exige código manual

Coleção (Collection)

- Tamanho dinâmico: cresce e diminui
- Métodos prontos e eficientes
- Organiza, armazena e recupera dados
- Reduz o tempo de desenvolvimento

- Coleção é um objeto que agrupa vários elementos em uma única unidade. Você usa a estrutura sem precisar saber como ela armazena os dados internamente.

CONCEITO

A API de coleções do Java

A API do Java fornece várias estruturas de dados predefinidas — as coleções — utilizadas para armazenar grupos de objetos relacionados.

Interfaces

Definem o contrato: List, Set, Queue, Map

Implementações

Classes concretas: ArrayList, HashSet, HashMap

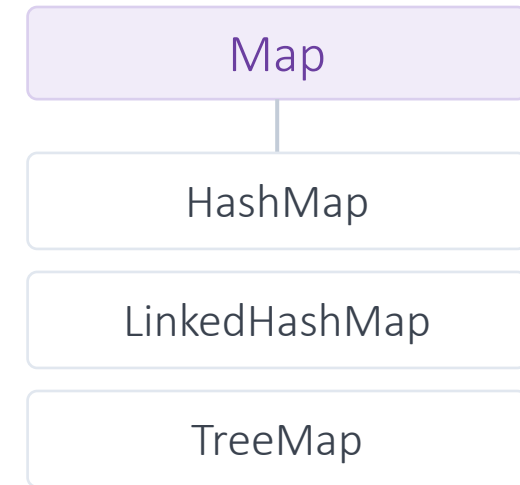
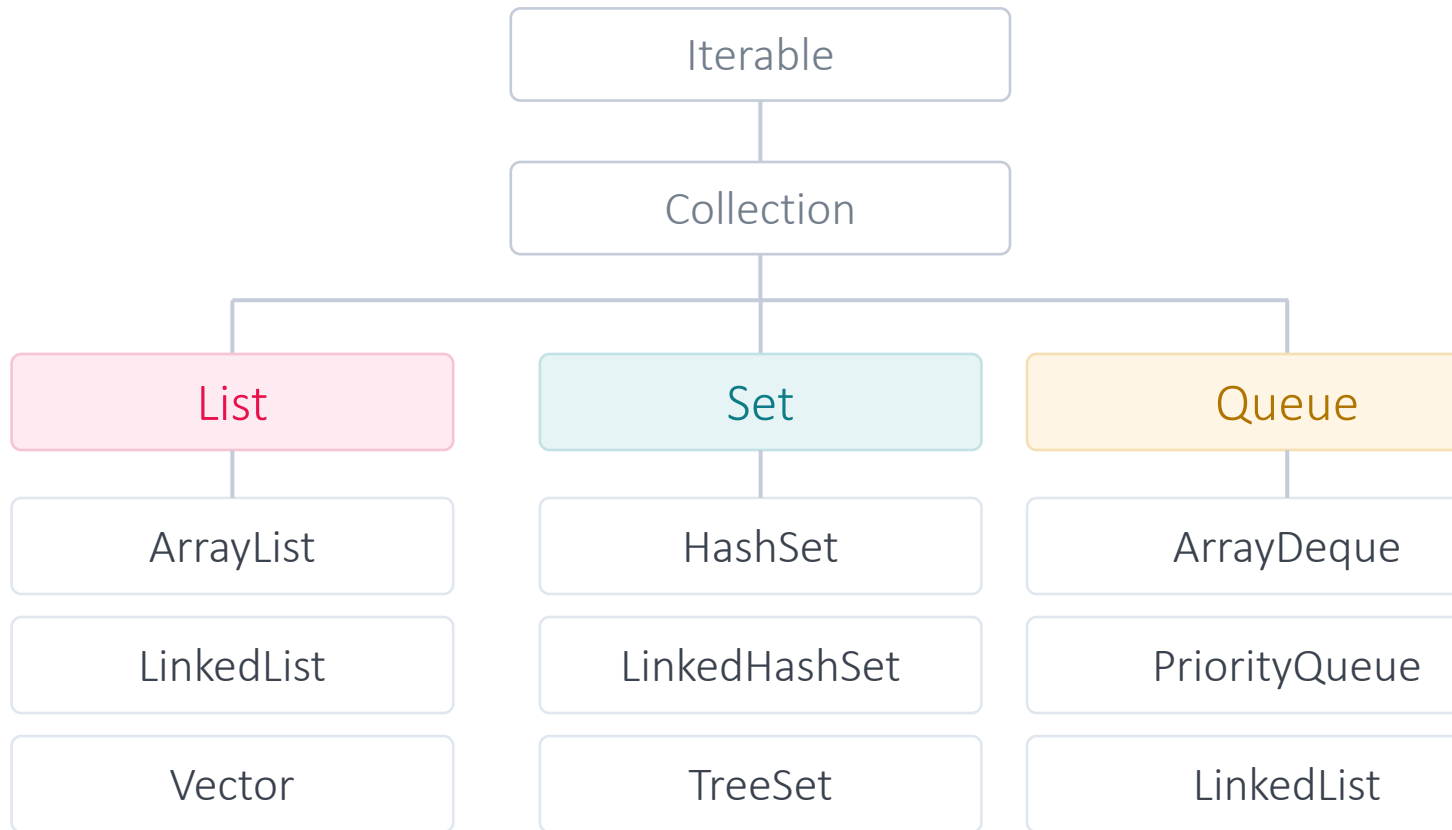
Algoritmos

Métodos utilitários:
Collections.sort(), reverse()

```
import java.util.*; // List, ArrayList, Set, HashSet, Map, HashMap...
```

- Todas as coleções vivem no pacote java.util — não esqueça o import.

Hierarquia do Collections Framework



- Map não estende Collection: trabalha com pares chave → valor.

As famílias: Set e List

Set / SortedSet Conjunto

- Modela a abstração matemática de conjunto
- Não mantém indexação nem contagem
- Não há elementos repetidos
- Pode ser ordenado (SortedSet) ou não

```
Set<String> s = new HashSet<>();
```

List Sequência indexada

- Coleção indexada, chamada de sequência
- Índices começam em zero, como nos vetores
- Métodos por posição, busca e remoção
- Percurso sequencial com ListIterator

```
List<String> l = new ArrayList<>();
```

As famílias: Queue e Map

Q Queue Fila

- Mantém uma fila em ordem de chegada
- Itens processados na ordem de entrada
- Lista FIFO: First-In, First-Out



M Map / SortedMap Mapa

- Armazena itens de chave e valor
- As chaves não podem ser duplicadas
- A chave localiza o valor associado
- As chaves podem ser ordenadas (SortedMap)

```
Map<Integer, Aluno> m =  
    new HashMap<>();
```

Interface List

É uma coleção ordenada, semelhante à ideia de um array. Também chamada de sequência e pode conter elementos duplicados.

ArrayList

Ideal para pesquisa randômica (acesso por índice).

```
lista.get(4);
```

LinkedList

Ideal para pesquisa sequencial e inserção no início.

```
lista.addFirst(x);
```

- As duas implementam List — o código que usa a lista continua igual. Muda apenas o objeto criado com `new`.

Classe ArrayList<T>

A classe ArrayList<T> (pacote java.util) resolve o problema dos arrays estáticos: ela altera dinamicamente o seu tamanho para acomodar mais elementos.

O T é um espaço reservado, substituído pelo tipo do elemento armazenado.

■ Declarações.java

```
1 List<String> lista = new ArrayList<>();  
2 ArrayList<String> lista = new ArrayList<>();  
3 ArrayList<Aluno> lista = new ArrayList<>();  
4 Collections<String> lista = new ArrayList();
```

- Prefira declarar pela interface: List<String>.

- A última linha (número 4) não compila: Collections é a classe utilitária de algoritmos, não uma interface de coleção.

ArrayList: principais métodos

Método	Descrição
<code>add(objeto)</code>	Adiciona um elemento ao fim do ArrayList.
<code>clear()</code>	Remove todos os elementos do ArrayList.
<code>contains(objeto)</code>	Retorna true se o elemento existir na lista.
<code>get(índice)</code>	Retorna o elemento no índice informado.
<code>indexOf(objeto)</code>	Índice da primeira ocorrência do elemento (inicia em zero).
<code>remove(índice)</code>	Remove o elemento na posição informada.
<code>remove(objeto)</code>	Remove a primeira ocorrência do objeto informado.
<code>size()</code>	Retorna o número de elementos armazenados.
<code>trimToSize()</code>	Ajusta a capacidade ao número atual de elementos.

Exemplo: a classe Aluno

■ Aluno.java

```
1 public class Aluno {  
2     private int rm;  
3     private String nome;  
4  
5     public Aluno(int rm, String nome) {  
6         this.rm = rm;  
7         this.nome = nome;  
8     }  
9     @Override  
10    public String toString() {  
11        return rm + " - " + nome;  
12    }  
13 }
```

- Atributos privados: encapsulamento.
- Construtor recebe os dados do objeto.
- **toString()** define como o objeto será impresso.
- Sem **toString()**, o **println()** mostra algo como `Aluno@7852e922`.

● **@Override** avisa o compilador de que o método está sendo sobrescrito de `Object`.

Exemplo: usando o ArrayList

■ TesteAluno.java

```
1 import java.util.*;
2
3 public class TesteAluno {
4     public static void main(String[] args) {
5         List<Aluno> lista = new ArrayList<>();
6         lista.add(new Aluno(5252, "Maria"));
7         lista.add(new Aluno(2323, "Pedro"));
8
9         for (int k = 0; k < lista.size(); k++) {
10             Aluno a = lista.get(k);
11             System.out.println(a);
12         }
13     }
14 }
```

● Run — saída no console

5252 - Maria

2323 - Pedro

- **add()** insere no fim da lista.
- **size()** devolve a quantidade atual.
- **get(k)** recupera pelo índice, que começa em 0.

LISTAS

Quatro formas de percorrer uma lista

1 for tradicional

```
int total = lista.size();  
for (int i = 0; i < total; i++) {  
    System.out.println(lista.get(i));  
}
```

2 for each

```
for (Aluno aluno : lista) {  
    System.out.println(aluno);  
}
```

3 Iterator

```
Iterator<Aluno> it = lista.iterator();  
while (it.hasNext()) {  
    System.out.println(it.next());  
}
```

4 lambda — Java 8

```
lista.forEach(aluno -> {  
    System.out.println(aluno);  
});
```

Ordenando listas

Como colocar a lista abaixo em ordem alfabética?

■ Ordenacao.java

```
1 List<String> lista = new ArrayList<>();
2 lista.add("Marcos");
3 lista.add("Beatriz");
4 lista.add("Zileide");
5 lista.add("Ana");
6
7 System.out.println(lista);
```

● Run — saída no console
[Marcos, Beatriz,
Zileide, Ana]

● A lista mantém a ordem de inserção. O **toString()** do ArrayList imprime os elementos entre colchetes.

A classe utilitária Collections

A classe **Collections** (pacote `java.util`) reúne métodos estáticos que operam sobre coleções — invocados pelo nome da classe, sem criar objetos.

```
static <T extends Comparable<? super T>> void sort(List<T> list)
```

Ordena a lista em ordem crescente, segundo a ordem natural dos seus elementos.

```
Collections.sort(lista);  
System.out.println(lista);
```

- Run — saída no console
[Ana, Beatriz, Marcos, Zileide]

- String já implementa **Comparable** — por isso a ordenação funciona sem escrever nada.

Mudando o critério de ordenação

O método `sort()` é sobrecarregado e aceita um segundo parâmetro: o critério de ordenação, obtido a partir da interface **Comparator** (pacote `java.util`).

```
static <T> void sort(List<T> list, Comparator<? super T> c)
```

```
Collections.sort(lista,  
                Comparator.reverseOrder());  
System.out.println(lista);
```

● Run — saída no console
[Zileide, Marcos,
Beatriz, Ana]

- **Comparable** define a ordem natural da classe. **Comparator** define ordens alternativas, sem alterar a classe.

ORDENAÇÃO

E para ordenar objetos?

Como ordenar uma lista de objetos Aluno em ordem alfabética?

■ Aluno.java

```
1 public class Aluno {  
2     private int rm;  
3     private String nome;  
4  
5     // construtor e getters  
6 }
```

```
List<Aluno> lista =  
    new ArrayList<>();  
lista.add(new Aluno(56, "Gustavo"));  
lista.add(new Aluno(32, "Pedro"));  
  
Collections.sort(lista);
```

● Erro de compilação

The method `sort(List<T>)` in the type `Collections` is not applicable for the arguments `(List<Aluno>)`

ORDENAÇÃO

A interface Comparable<T>

O método **compareTo()** faz parte da interface **Comparable<T>** e define a ordem natural entre objetos de uma classe. É o mecanismo usado por **TreeSet**, **TreeMap**, **Collections.sort()** e **Arrays.sort()**.

```
public int compareTo(T outroObjeto) { ... }
```

O retorno é um número inteiro:

< 0

o objeto atual é menor

= 0

os objetos são iguais na
ordenação

> 0

o objeto atual é maior



As classes wrapper e **String** já implementam **Comparable**: "banana".compareTo("abacaxi") > 0.

compareTo(): pontos de atenção

1

Consistência com equals()

Não é obrigatório, mas é fortemente recomendado que `a.compareTo(b) == 0` implique `a.equals(b) == true`.

2

Critério bem definido

Evite comparações arbitrárias: ordene por nome, por idade, por matrícula — algo que faça sentido no domínio.

3

Comparable × Comparator

Comparable define a ordem natural da classe. **Comparator** permite ordens alternativas sem modificar a classe.

ORDENAÇÃO

Aluno implements Comparable<Aluno>

■ Aluno.java

```
1 public class Aluno
2     implements Comparable<Aluno> {
3     private int rm;
4     private String nome;
5
6     @Override
7     public int compareTo(Aluno aluno) {
8         return this.nome
9             .compareTo(aluno.getNome());
10    }
11 }
```

● Run — saída no console
[Aluno@7852e922,
Aluno@4e25154f,
Aluno@70dea4e]

● A lista foi ordenada, mas o método **toString()** ainda não foi sobrescrito — por isso o console mostra o endereço do objeto.

Sobrescrevendo toString()

■ Aluno.java

```
1 public class Aluno implements Comparable<Aluno> {
2     private int rm;
3     private String nome;
4
5     @Override
6     public String toString() {
7         return rm + " - " + nome;
8     }
9     @Override
10    public int compareTo(Aluno a) {
11        return this.nome.compareTo(a.getNome());
12    }
13 }
```

● Run — saída no console

```
[89 - Beatriz,
15 - Gabriela,
56 - Gustavo,
32 - Pedro]
```

● Agora a ordem alfabética por nome fica evidente na saída.

ORDENAÇÃO

Mudando o critério: ordenar por rm

■ Aluno.java

```
1 @Override
2 public int compareTo(Aluno aluno) {
3     if (this.rm > aluno.getRm()) {
4         return 1;
5     }
6     else if (this.rm < aluno.getRm()) {
7         return -1;
8     }
9     return 0;
10 }
```

Versão equivalente, mais curta:

```
return Integer.compare(this.rm, aluno.getRm());
```

● Run — saída no console

```
[15 - Gabriela,
32 - Pedro,
56 - Gustavo,
89 - Beatriz]
```

● Basta alterar o **compareTo()** — o código que chama **Collections.sort()** não muda.

Vários critérios: Comparator do Java 8

Quando a classe tem vários atributos, é possível escolher o critério na hora de ordenar, usando recursos da versão 8 da linguagem.

```
lista.sort(Comparator.comparing(Aluno::getNome));
```

Comparator

a interface que define o critério de comparação

comparing(...)

gera o comparador a partir de um atributo

Aluno::getNome

referência de método: método getNome da classe Aluno

```
lista.sort(Comparator.comparing(Aluno::getNome)  
            .thenComparing(Aluno::getRm)); // critério de desempate
```

IGUALDADE

Comparando dois objetos

Como verificar se dois objetos Aluno são iguais? O operador `==` compara referências, e não o conteúdo.

■ Teste.java

```
1 Aluno aluno1 = new Aluno(12, "Ana Beatriz");
2 Aluno aluno2 = new Aluno(12, "Ana Beatriz");
3
4 if (aluno1 == aluno2) {
5     System.out.println("Mesmo aluno");
6 }
7 else {
8     System.out.println("Alunos diferentes");
9 }
```

● Run — saída no console
Alunos diferentes

● São dois objetos distintos na memória. O `==` pergunta se as duas variáveis apontam para o mesmo objeto.

IGUALDADE

Primeira alternativa: um método próprio

Dois objetos Aluno são iguais quando têm os mesmos atributos. Uma alternativa é criar um método que compare os atributos dos dois objetos.

■ Aluno.java

```
1 public static boolean comparar(Aluno aluno1, Aluno aluno2) {  
2     boolean iguais = false;  
3     if (aluno1.getRm() == aluno2.getRm()  
4         && aluno1.getNome().equals(aluno2.getNome())) {  
5         iguais = true;  
6     }  
7     return iguais;  
8 }
```

- Funciona, mas é um método fora do padrão: coleções e frameworks não sabem que ele existe.

IGUALDADE

Segunda alternativa: sobrescrever equals()

■ Aluno.java

```
1 @Override
2 public boolean equals(Object o) {
3     if ((o instanceof Aluno)
4         && ((Aluno) o).getRm() == this.rm) {
5         return true;
6     }
7     return false;
8 }
```

```
if (aluno1.equals(aluno2)) {
    System.out.println("Mesmo aluno");
}
```

● Run — saída no console
Mesmo aluno

● equals() é o padrão da linguagem: está definido em Object e é usado por contains(), indexOf(), HashSet e HashMap.

ArrayList ou LinkedList?

Para a lista de aulas da classe Disciplina temos duas opções. Existe diferença de desempenho entre elas?

■ Disciplina.java

```
1 public class Disciplina {  
2     private String nome;  
3     private String professor;  
4     private List<Aula> listaAula;  
5 }
```

■ Aula.java

```
1 public class Aula {  
2     private String conteudo;  
3     private int duracao;  
4 }
```

As duas declarações abaixo são válidas:

```
private List<Aula> listaAula = new ArrayList<>();  
private List<Aula> listaAula = new LinkedList<>();
```

Qual a diferença? Desempenho

ArrayList

Usa internamente um array.

- Acesso rápido por índice com **get()**
- Inserção e remoção no início são lentas
- Todos os elementos precisam ser deslocados

LinkedList

Usa uma lista ligada.

- Ótimo desempenho para inserir no início
- Busca aleatória é lenta
- É preciso percorrer os elementos um a um

Operação	ArrayList	LinkedList
get(indice)	rápido	lento
add no fim	rápido	rápido
add no início	lento	rápido

CONJUNTOS

Interface Set<>

List é a interface mais usada, através da classe **ArrayList**. A segunda mais utilizada é **Set**, implementada pela classe concreta **HashSet**.

■ TesteSet.java

```
1 Set<String> listaAlunos = new HashSet<>();  
2 listaAlunos.add("Selmini");  
3 listaAlunos.add("Ana");  
4 listaAlunos.add("Beatriz");  
5 listaAlunos.add("Maria");  
6 listaAlunos.add("Carlos");  
7  
8 System.out.println(listaAlunos);
```

● Run — saída no console
[Selmini, Ana,
Maria, Carlos,
Beatriz]

● Não existe garantia da ordem em que os elementos foram inseridos.

CONJUNTOS

Set: quais são as vantagens?

Não permite elementos duplicados

Busca de um elemento é mais rápida que em **ArrayList**

■ TesteSet.java

```
1 Set<String> listaAlunos = new HashSet<>();
2 listaAlunos.add("Selmini");
3 listaAlunos.add("Ana");
4 listaAlunos.add("Beatriz");
5 listaAlunos.add("Selmini");
6 listaAlunos.add("Carlos");
7
8 System.out.println(listaAlunos);
```

● Run — saída no console
[Selmini, Ana,
Carlos, Beatriz]

● Elementos repetidos são simplesmente ignorados.

Usando Set na classe Disciplina

Disciplina.java

```
1 public class Disciplina {  
2     private String nome;  
3     private List<Aula> listaAula = new ArrayList<>();  
4     private Set<Aluno> listaAluno = new HashSet<>();  
5  
6     public void matricular(Aluno aluno) {  
7         listaAluno.add(aluno);  
8     }  
9  
10    public boolean estaMatriculado(Aluno aluno) {  
11        return listaAluno.contains(aluno);  
12    }  
13 }
```

● A lista de aulas continua sendo um **ArrayList**.

● O **Set** garante que o mesmo aluno não seja matriculado duas vezes.

CONJUNTOS

contains() usa equals()

Como verificar se um aluno está matriculado usando apenas o seu nome? O método **contains()** da interface **Collection** utiliza **equals()** para fazer a comparação.

■ Aluno.java

```
1 @Override
2 public boolean equals(Object aluno) {
3     Aluno aux = (Aluno) aluno;
4     if (this.nome.equalsIgnoreCase(aux.getNome())
5         && aux.getRm() == this.rm) {
6         return true;
7     }
8     return false;
9 }
```



Cuidado: se o nome for null, uma exceção será lançada.



Sobrescrever **equals()** muda o comportamento de **contains()**, **indexOf()** e do **HashSet**.

Sobrescrevendo hashCode()

Para uma busca realmente eficiente em um **Set** é necessário sobrescrever também o método **hashCode()**.

■ Aluno.java

```
1 @Override
2 public int hashCode() {
3     return this.nome.hashCode();
4 }
```

- 1 O HashSet calcula o hashCode() do objeto
- 2 Usa o valor para escolher o bucket
- 3 Compara com equals() só dentro daquele bucket

- Regra de ouro: sempre que sobrescrever **equals()**, sobrescreva **hashCode()**. Objetos iguais devem ter o mesmo **hashCode** — caso contrário o **HashSet** e o **HashMap** deixam de funcionar corretamente.

Interface Map

Como localizar um aluno pelo seu rm? Se a quantidade de alunos for grande e a busca for frequente, percorrer a coleção inteira tem custo computacional alto.

- Solução: utilizar a estrutura **Map**, que associa uma chave a um valor.

■ Disciplina.java

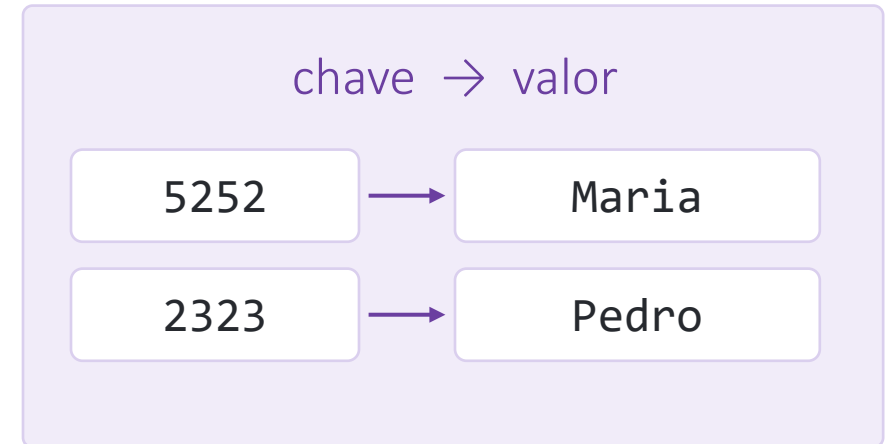
```
1 public class Disciplina {  
2     private String nome;  
3     private String professor;  
4     private List<Aula> listaAula = new ArrayList<>();  
5     private Set<Aluno> listaAluno = new HashSet<>();  
6     private Map<Integer, Aluno> listaMatricula = new HashMap<>();  
7 }
```

Inserindo valores em um Map

O método **put()** recebe a chave e o valor a ser armazenado.

■ Disciplina.java

```
1 private Map<Integer, Aluno> listaMatricula
2     = new HashMap<>();
3
4 public void matricular(Aluno aluno) {
5     listaAluno.add(aluno);
6     listaMatricula.put(aluno.getRm(), aluno);
7 }
```



- Se a chave já existir, **put()** substitui o valor anterior — chaves não se repetem em um **Map**.

Buscando um valor em um Map

A busca é feita pelo método **get()**, informando a chave.

■ Disciplina.java

```
1 private Map<Integer, Aluno> listaMatricula
2     = new HashMap<>();
3
4 public Aluno localizarAluno(int rm) {
5     return listaMatricula.get(rm);
6 }
```

```
boolean existe = listaMatricula.containsKey(rm);
```



A busca pela chave é praticamente imediata, independente do tamanho do mapa.



Se a chave não existir, **get()** devolve **null** — trate esse caso antes de usar o objeto.

Percorrendo um Map

■ Teste2.java

```
1 import java.util.*;
2
3 public class Teste2 {
4     public static void main(String[] args) {
5         Map<Integer, Aluno> lista = new HashMap<>();
6         lista.put(1, new Aluno(1, "Ana"));
7         lista.put(2, new Aluno(2, "Pedro"));
8
9         Set<Integer> listaRm = lista.keySet();
10        for (Integer rm : listaRm) {
11            System.out.println(lista.get(rm));
12        }
13    }
14 }
```

- **keySet()** retorna um **Set** contendo todas as chaves do **Map**.

- Alternativa mais direta:
lista.forEach((rm, aluno)
-> ...)

EXERCÍCIO 01

Ranking de candidatos

Você foi contratado para auxiliar no processo seletivo de uma empresa. Dada uma lista de candidatos, gere um ranking ordenado pelos critérios abaixo.

1

Nota técnica (decimal)
decrecente

2

Anos de experiência
decrecente

3

Nome
crescente — desempate final

- Depois de ordenar, imprima o ranking usando for each, mostrando a posição (1º, 2º, ...), nome, nota e anos de experiência.

```
// dica: Comparator.comparing(...).reversed().thenComparing(...)
```

EXERCÍCIO 02

Gerenciador de matrículas

Você recebeu duas listas de alunos matriculados em disciplinas diferentes: Estruturas de Dados e Banco de Dados. Cada aluno tem rm, nome e curso.

- 1 Eliminar duplicidades com HashSet — dois alunos com o mesmo rm são o mesmo aluno.
- 2 Calcular a união: alunos em pelo menos uma das disciplinas.
- 3 Calcular a interseção: alunos presentes nas duas disciplinas.
- 4 Calcular a diferença: só de Estruturas e só de Banco.
- 5 Verificar a pertinência de um determinado rm.
- 6 Exibir os resultados ordenados por nome apenas na impressão.

- A coleção principal continua sendo um **HashSet** — a ordenação acontece apenas na hora de imprimir.

EXERCÍCIO 03

Contador de letras

Escreva um programa em Java que leia uma frase informada via teclado e conte a frequência de cada uma das letras presentes na frase.

■ ContadorLetras.java

```
1 Scanner sc = new Scanner(System.in);
2 System.out.print("Frase: ");
3 String frase = sc.nextLine().toLowerCase();
4
5 Map<Character, Integer> mapa = new HashMap<>();
6
7 // percorra a frase e atualize o mapa
```

```
mapa.put(c, mapa.getDefault(c, 0) + 1);
```

● Exemplo de saída

Frase: java

a = 2

j = 1

v = 1

● Dica: **getDefault()** evita testar se a letra já existe no mapa.

Record

Record é um tipo de classe imutável e orientada a dados, criado para declarar carregadores de dados com muito menos código e sem surpresas em `equals`, `hashCode` e `toString`.

Carregador de dados

Objetos cujo propósito é representar e transportar dados — típicos DTOs entre camadas e serviços.

Sintaxe curta

Declara apenas os componentes. Acessores, construtor, `equals`, `hashCode` e `toString` são gerados automaticamente.

Sempre final

Todo record é final por padrão da linguagem: não é possível estender um record.

```
public record Cliente(long cpf, String nome, String email) { }
```

Record na prática

■ Main.java

```

1 public record Cliente(long cpf, String nome,
2                       String email) { }
3
4 public class Main {
5     public static void main(String[] args) {
6         Cliente c = new Cliente(12345L, "Joao",
7                                "joao@gmail.com");
8         System.out.println(c.cpf());
9         System.out.println(c);
10    }
11 }

```

● Run — saída no console

```

12345
Cliente[cpf=12345,
nome=Joao,
email=joao@gmail.com]

```

● Os acessores têm o nome do componente: `c.cpf()`, e não `getCpf()`.

Record: construtor compacto

■ Cliente.java

```

1 public record Cliente(long cpf, String nome,
2                       String email) {
3     public Cliente {
4         if (cpf < 0 || cpf > 99_999_999_999L)
5             throw new IllegalArgumentException(
6                 "CPF inválido.");
7         if (nome == null || nome.isBlank())
8             throw new IllegalArgumentException(
9                 "Nome inválido.");
10
11         nome = nome.trim();
12         email = email.trim().toLowerCase();
13     }
14 }

```

● **isBlank():** true se a String está vazia ou só tem espaços em branco — espaço, tab, quebra de linha.

● **trim():** remove os espaços do início e do fim e devolve uma nova String — a original não muda, pois String é imutável.

Record: construtor canônico

■ Cliente.java

```
1 public record Cliente(long cpf, String nome,  
2                       String email) {  
3     public Cliente(long cpf, String nome,  
4                   String email) {  
5         if (cpf < 0 || cpf > 99_999_999_999L)  
6             throw new IllegalArgumentException(  
7                 "CPF inválido.");  
8  
9         this.cpf    = cpf;  
10        this.nome   = nome.trim();  
11        this.email  = email.trim().toLowerCase();  
12    }  
13 }
```

● Aqui a assinatura completa é escrita e a atribuição dos campos com **this** fica por sua conta.

● O construtor compacto do slide anterior faz o mesmo com menos código — prefira essa forma.

Resumo: qual coleção usar?

Preciso de...	Use	Porque
Ordem de inserção e acesso por índice	<code>ArrayList</code>	acesso rápido por posição
Muitas inserções e remoções no início	<code>LinkedList</code>	não desloca elementos
Elementos únicos, sem repetição	<code>HashSet</code>	ignora duplicatas
Busca por uma chave própria	<code>HashMap</code>	acesso direto pela chave
Coleção sempre ordenada	<code>TreeSet</code> / <code>TreeMap</code>	mantém a ordem natural

- Declare sempre pela interface (**List**, **Set**, **Map**) e troque apenas a implementação quando o desempenho exigir.

Referências

1

DEITEL, H. M.; DEITEL, P. J. Java: como programar.
10ª edição. São Paulo: Prentice-Hall, 2010.

2

SCHILDT, H. Java para iniciantes.
6ª edição. Porto Alegre: Bookman, 2015.

3

KNUDSEN, J.; NIEMEYER, P. Aprendendo Java.
Rio de Janeiro: Elsevier Campus, 2000.

Referências

1

FLANAGAN, D. Java: o guia essencial.

Porto Alegre: Bookman, 2006.

2

ARNOLD, K.; GOSLING, J.; HOLMES, D. The Java programming language.

4th edition. Addison-Wesley, 2005.

3

JANDL JUNIOR, P. Introdução ao Java.

São Paulo: Berkeley, 2002.